

Installation

```
!pip install -q torch datasets transformers
```

Imports

```
import os
from collections import OrderedDict
import torch
import torch.nn as nn
import math
from torch.utils.data import DataLoader
from datasets import load_dataset
from transformers import AutoTokenizer
from torch.nn.utils.rnn import pad_sequence

from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive
```

Data Preparation

```
# We use an English-French dataset from OPUS
dataset = load_dataset("opus_books", "en-fr", split="train[:100000]")

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.

{"model_id": "6e7d6166d33340759bf809352767cba5", "version_major": 2, "version_minor": 0}
```

```

{"model_id": "39da45b9f11c4ff5a5420e6e99191820", "version_major": 2, "version_minor": 0}

{"model_id": "ca77602378724176ac09be2a9b210949", "version_major": 2, "version_minor": 0}

tokenizer_en = AutoTokenizer.from_pretrained("bert-base-uncased")
tokenizer_fr = AutoTokenizer.from_pretrained("dbmdz/bert-base-french-europeana-cased")

{"model_id": "59731b1d84f249f2b0bbaaa91613acfc", "version_major": 2, "version_minor": 0}

{"model_id": "ffd36b531d254c4d97b39ee4c3f04415", "version_major": 2, "version_minor": 0}

{"model_id": "87afe91414814c9e9d2de22e42b47bed", "version_major": 2, "version_minor": 0}

{"model_id": "70099dc402a24517b095b4ce5d236551", "version_major": 2, "version_minor": 0}

{"model_id": "e6462440d9834293af324e9661f95de1", "version_major": 2, "version_minor": 0}

{"model_id": "ec2b62b9f3c64778a20486b032555840", "version_major": 2, "version_minor": 0}

{"model_id": "c7e7429c65594fb38f71e63b3aa7308d", "version_major": 2, "version_minor": 0}

SRC_LANGUAGE = 'en'
TGT_LANGUAGE = 'fr'
BOS_IDX = tokenizer_fr.cls_token_id or 101 # [CLS] acting as Start of Sentence
EOS_IDX = tokenizer_fr.sep_token_id or 102 # [SEP] acting as End of Sentence
PAD_IDX = tokenizer_fr.pad_token_id or 0 # [PAD]

MAX_SEQ_LEN = 150 # Safe limit for Colab T4 GPUs

def collate_fn(batch):
    src_batch, tgt_batch = [], []
    for item in batch:
        src_text = item['translation'][SRC_LANGUAGE]
        tgt_text = item['translation'][TGT_LANGUAGE]

        # Add truncation and max_length here!
        src_tokens = tokenizer_en.encode(src_text,
            add_special_tokens=True, truncation=True, max_length=MAX_SEQ_LEN)
        tgt_tokens = tokenizer_fr.encode(tgt_text,
            add_special_tokens=True, truncation=True, max_length=MAX_SEQ_LEN)

```

```

        src_batch.append(torch.tensor(src_tokens))
        tgt_batch.append(torch.tensor(tgt_tokens))

    src_batch = pad_sequence(src_batch, padding_value=PAD_IDX)
    tgt_batch = pad_sequence(tgt_batch, padding_value=PAD_IDX)
    return src_batch, tgt_batch

batch_size = 32          # Drop this from 64 to 32
dataloader = DataLoader(
    dataset,
    batch_size=batch_size,
    collate_fn=collate_fn,
    shuffle=True,
    num_workers=2,        # 2 is the sweet spot for Colab's 2-
                           core CPU
    pin_memory=True,
    persistent_workers=False # <-- Change this to False to save
                             System RAM
)

```

Model Definition

```

class PositionalEncoding(nn.Module):
    def __init__(self, emb_size: int, dropout: float, maxlen: int =
5000):
        super(PositionalEncoding, self).__init__()
        den = torch.exp(-torch.arange(0, emb_size, 2)* math.log(10000)
/ emb_size)
        pos = torch.arange(0, maxlen).reshape(maxlen, 1)
        pos_embedding = torch.zeros((maxlen, emb_size))
        pos_embedding[:, 0::2] = torch.sin(pos * den)
        pos_embedding[:, 1::2] = torch.cos(pos * den)
        pos_embedding = pos_embedding.unsqueeze(-2)

        self.dropout = nn.Dropout(dropout)
        self.register_buffer('pos_embedding', pos_embedding)

    def forward(self, token_embedding: torch.Tensor):
        # Add positional encoding to token embeddings
        return self.dropout(token_embedding +
self.pos_embedding[:token_embedding.size(0), :])

class TokenEmbedding(nn.Module):
    def __init__(self, vocab_size: int, emb_size):
        super(TokenEmbedding, self).__init__()
        self.embedding = nn.Embedding(vocab_size, emb_size)
        self.emb_size = emb_size

```

```

        def forward(self, tokens: torch.Tensor):
            return self.embedding(tokens.long()) *
math.sqrt(self.emb_size)

# The Main Transformer Model
class Seq2SeqTransformer(nn.Module):
    def __init__(self, num_encoder_layers: int, num_decoder_layers:
int,
                    emb_size: int, nhead: int, src_vocab_size: int,
tgt_vocab_size: int,
                    dim_feedforward: int = 512, dropout: float = 0.1):
        super(Seq2SeqTransformer, self).__init__()

        # PyTorch's built-in Transformer module
        self.transformer = nn.Transformer(d_model=emb_size,
nhead=nhead,
num_encoder_layers=num_encoder_layers,
num_decoder_layers=num_decoder_layers,
dim_feedforward=dim_feedforward,
                    dropout=dropout)
        self.generator = nn.Linear(emb_size, tgt_vocab_size)
        self.src_tok_emb = TokenEmbedding(src_vocab_size, emb_size)
        self.tgt_tok_emb = TokenEmbedding(tgt_vocab_size, emb_size)
        self.positional_encoding = PositionalEncoding(emb_size,
dropout=dropout)

    def forward(self, src: torch.Tensor, trg: torch.Tensor, src_mask:
torch.Tensor,
                    tgt_mask: torch.Tensor, src_padding_mask:
torch.Tensor,
                    tgt_padding_mask: torch.Tensor,
memory_key_padding_mask: torch.Tensor):

        src_emb = self.positional_encoding(self.src_tok_emb(src))
        tgt_emb = self.positional_encoding(self.tgt_tok_emb(trg))

        # Pass through the standard PyTorch Transformer
        outs = self.transformer(src_emb, tgt_emb, src_mask, tgt_mask,
None,
                    src_padding_mask, tgt_padding_mask,
memory_key_padding_mask)
        return self.generator(outs)

```

MModel Generation

```
def generate_square_subsequent_mask(sz):
    # Creates a boolean matrix where 'True' means "mask this out"
    (future words)
    # and 'False' means "allowed to look at" (past words).
    mask = torch.triu(torch.ones(sz, sz), diagonal=1).bool()
    return mask

def create_mask(src, tgt):
    src_seq_len = src.shape[0]
    tgt_seq_len = tgt.shape[0]

    tgt_mask =
generate_square_subsequent_mask(tgt_seq_len).to(src.device)
    # Ensure src_mask is explicitly a boolean tensor
    src_mask = torch.zeros((src_seq_len, src_seq_len),
dtype=torch.bool).to(src.device)

    src_padding_mask = (src == PAD_IDX).transpose(0, 1)
    tgt_padding_mask = (tgt == PAD_IDX).transpose(0, 1)

    return src_mask, tgt_mask, src_padding_mask, tgt_padding_mask
```

Training Setup

```
if torch.cuda.is_available():
    DEVICE = torch.device('cuda') # NVIDIA GPUs
elif torch.backends.mps.is_available():
    DEVICE = torch.device('mps') # Apple Silicon (M1/M2/M3)
else:
    DEVICE = torch.device('cpu') # Fallback
print(f"Training on: {DEVICE}")

SRC_VOCAB_SIZE = tokenizer_en.vocab_size
TGT_VOCAB_SIZE = tokenizer_fr.vocab_size
EMB_SIZE = 128 # Equivalent to d_model in Keras
NHEAD = 8
FFN_HID_DIM = 512 # Equivalent to dff in Keras
NUM_ENCODER_LAYERS = 4
NUM_DECODER_LAYERS = 4

model = Seq2SeqTransformer(NUM_ENCODER_LAYERS, NUM_DECODER_LAYERS,
EMB_SIZE,
NHEAD, SRC_VOCAB_SIZE, TGT_VOCAB_SIZE,
FFN_HID_DIM).to(DEVICE)

Training on: cuda
```

```

save_path = "/content/drive/MyDrive/Model_Checkpoints/"

# Automatically create the folder if it does not exist!
os.makedirs(save_path, exist_ok=True)
checkpoint_file = f"{save_path}transformer_epoch_49.pt"

# 1. Load the raw dictionary from the file
state_dict = torch.load(checkpoint_file, map_location=DEVICE,
weights_only=True)

# 2. Create a new dictionary to hold the cleaned names
cleaned_state_dict = OrderedDict()

# 3. Loop through and strip the "_orig_mod." prefix
for key, value in state_dict.items():
    clean_key = key.replace('_orig_mod.', '')
    cleaned_state_dict[clean_key] = value

# 4. Load the cleaned weights into your model
model.load_state_dict(cleaned_state_dict)

print(f"Successfully loaded weights from {checkpoint_file}!")

Successfully loaded weights from
/content/drive/MyDrive/Model_Checkpoints/transformer_epoch_49.pt!

model = torch.compile(model)

loss_fn = torch.nn.CrossEntropyLoss(ignore_index=PAD_IDX)
optimizer = torch.optim.Adam(model.parameters(), lr=0.0001,
betas=(0.9, 0.98), eps=1e-9)

```

Trainign Loop

```

accumulation_steps = 8 # Simulates a batch size 4x larger

def train_epoch(model, dataloader, optimizer, scaler):
    model.train()
    losses = 0
    optimizer.zero_grad(set_to_none=True)

    for i, (src, tgt) in enumerate(dataloader):
        src, tgt = src.to(DEVICE), tgt.to(DEVICE)
        tgt_input, tgt_out = tgt[:-1, :], tgt[1:, :]
        src_mask, tgt_mask, src_padding_mask, tgt_padding_mask =
create_mask(src, tgt_input)

        with torch.amp.autocast('cuda'):
            logits = model(src, tgt_input, src_mask, tgt_mask,

```

```

src_padding_mask, tgt_padding_mask, src_padding_mask)
    loss = loss_fn(logits.reshape(-1, logits.shape[-1]),
tgt_out.reshape(-1))

    # Divide loss by accumulation steps
    loss = loss / accumulation_steps

    scaler.scale(loss).backward()

    # Only step the optimizer every 4 batches
    if (i + 1) % accumulation_steps == 0:
        scaler.step(optimizer)
        scaler.update()
        optimizer.zero_grad(set_to_none=True)

    losses += loss.item() * accumulation_steps # Un-divide for
accurate tracking

    return losses / len(dataloader)

scaler = torch.amp.GradScaler('cuda') # Use 'cuda' if NVIDIA, or
remove if using Apple Silicon
print("Starting training...")
for epoch in range(50, 100):
    train_loss = train_epoch(model, dataloader, optimizer, scaler)
    print(f"Epoch: {epoch}, Loss: {train_loss:.3f}")

    # Save the model weights after every epoch!
    torch.save(model.state_dict(),
f"{save_path}transformer_epoch_{epoch}.pt")

```

Starting training...

W0427 15:55:17.232000 1517 torch/_inductor/utils.py:1679] [1/0] Not enough SMs to use max_autotune_gemm mode

```

Epoch: 50, Loss: 3.598
Epoch: 51, Loss: 3.582
Epoch: 52, Loss: 3.568
Epoch: 53, Loss: 3.557
Epoch: 54, Loss: 3.544
Epoch: 55, Loss: 3.533
Epoch: 56, Loss: 3.522
Epoch: 57, Loss: 3.511
Epoch: 58, Loss: 3.500
Epoch: 59, Loss: 3.489
Epoch: 60, Loss: 3.478
Epoch: 61, Loss: 3.469
Epoch: 62, Loss: 3.458
Epoch: 63, Loss: 3.449
Epoch: 64, Loss: 3.438

```

```
Epoch: 65, Loss: 3.428
Epoch: 66, Loss: 3.420
Epoch: 67, Loss: 3.411
Epoch: 68, Loss: 3.402
Epoch: 69, Loss: 3.394
Epoch: 70, Loss: 3.384
Epoch: 71, Loss: 3.377
Epoch: 72, Loss: 3.367
Epoch: 73, Loss: 3.360
Epoch: 74, Loss: 3.352
Epoch: 75, Loss: 3.344
Epoch: 76, Loss: 3.337
Epoch: 77, Loss: 3.328
Epoch: 78, Loss: 3.321
Epoch: 79, Loss: 3.314
Epoch: 80, Loss: 3.307
Epoch: 81, Loss: 3.299
Epoch: 82, Loss: 3.293
Epoch: 83, Loss: 3.285
Epoch: 84, Loss: 3.276
Epoch: 85, Loss: 3.271
Epoch: 86, Loss: 3.265
Epoch: 87, Loss: 3.257
Epoch: 88, Loss: 3.252
Epoch: 89, Loss: 3.244
Epoch: 90, Loss: 3.239
Epoch: 91, Loss: 3.232
Epoch: 92, Loss: 3.226
Epoch: 93, Loss: 3.220
Epoch: 94, Loss: 3.215
Epoch: 95, Loss: 3.209
Epoch: 96, Loss: 3.202
Epoch: 97, Loss: 3.197
Epoch: 98, Loss: 3.192
Epoch: 99, Loss: 3.186
```

Verification

```
def translate_sentence(model, sentence, tokenizer_src, tokenizer_tgt,
device, max_len=50):
    model.eval() # Put the model in evaluation mode

    # 1. Tokenize the input sentence
    src_tokens = tokenizer_src.encode(sentence,
add_special_tokens=True)
    src_tensor = torch.tensor(src_tokens).unsqueeze(1).to(device) #
Shape: (seq_len, 1)
```



```

    # 2. Create the source mask
    src_mask = torch.zeros((src_tensor.shape[0],
src_tensor.shape[0])).type(torch.bool).to(device)

    with torch.no_grad():
        # 3. Encode the source sentence once
        src_emb =
model.positional_encoding(model.src_tok_emb(src_tensor))
        memory = model.transformer.encoder(src_emb, src_mask)

        # 4. Start the target sentence with the Start-of-Sequence
token
        tgt_tokens = [BOS_IDX]

        for i in range(max_len):
            tgt_tensor =
torch.tensor(tgt_tokens).unsqueeze(1).to(device)

            # Create the target causal mask to hide future words
            tgt_mask =
generate_square_subsequent_mask(tgt_tensor.size(0)).to(device)

            # 5. Decode the current sequence
            tgt_emb =
model.positional_encoding(model.tgt_tok_emb(tgt_tensor))
            out = model.transformer.decoder(tgt_emb, memory, tgt_mask)

            # Pass the output through the final linear layer
            logits = model.generator(out)

            # Get the token with the highest probability (greedy
decoding)
            next_word = logits[-1, 0, :].argmax().item()
            tgt_tokens.append(next_word)

            # Stop if we hit the End-of-Sequence token
            if next_word == EOS_IDX:
                break

        # 6. Convert token IDs back to a readable string
        translated_sentence = tokenizer_tgt.decode(tgt_tokens,
skip_special_tokens=True)
        return translated_sentence

# Test sentences (Keep them relatively simple for a small, quickly
trained model)
test_sentences = [
    "This is a good idea.",
    "The house is very big.",
    "I am reading a book.",

```

```

    "He is a tall man."
]

print("--- Model Validation Test ---")
for sentence in test_sentences:
    translation = translate_sentence(
        model=model,
        sentence=sentence,
        tokenizer_src=tokenizer_en,
        tokenizer_tgt=tokenizer_fr,
        device=DEVICE
    )
    print(f"English: {sentence}")
    print(f"French : {translation}")
    print("-" * 30)

```

```

--- Model Validation Test ---
English: This is a good idea.
French : C ' est un bon cœur.
-----
English: The house is very big.
French : La maison est très belle.
-----
English: I am reading a book.
French : Je suis un livre.
-----
English: He is a tall man.
French : Il est un homme très grand.
-----

```